

Lens-Based CGEI Algorithm with GPU Acceleration for Integral Imaging Light-Field Displays System

Shao-Yu Hung and Hoang-Yan Lin

Graduate Institute of Photonics and Optoelectronics, National Taiwan University, Taipei 106, Taiwan

Abstract

Automotive Augmented Reality (AR) displays demand low-latency elemental image generation for safety-critical cues. However, conventional pixel-wise Computer Generated Elemental Image (CGEI) methods are impractical for dynamic scenes due to high computational costs. We propose a dynamic synthesis pipeline integrating skeletonization-based trajectory extraction with a lens-based LUT framework. By utilizing a Unified Mask Stack and optimized PTX CUDA kernels, our method achieves single-pass multi-depth synthesis. Experiments show a generation time of just 0.032 s while maintaining high image quality (SSIM > 0.91), validating the system's potential for approaching real-time automotive AR.

Author Keywords

integral image; light field display; elemental image; real time; lens-based rendering; GPU.

1. Introduction

Automotive augmented reality (AR) systems require low-latency rendering for safety-critical scenarios; however, the computational bottleneck of pixel-wise backward ray tracing in traditional Computer-Generated Elemental Image (CGEI) methods limits the deployment of light-field displays. While inspired by prior camera multi-view acquisition research such as Zhu et al. [1], such approaches necessitate rendering a dense array of viewpoints, resulting in high latency insufficient for dynamic driving environments. To address this, we propose a lens-based CGEI method that utilizes precomputed geometric Look-Up Tables (LUTs)—handling scaling, flipping, and padding—to eliminate the prohibitive overhead of pixel-wise ray tracing. By employing a GPU-parallelized framework with a mask stack and optimized PTX CUDA kernels, we achieve single-pass multi-depth synthesis with a generation time of just 0.032 seconds, validating the system's potential for real-time deployment. In the display architecture proposed in this study, a freeform mirror is utilized to project light-field images, where dashboard information is imaged at a distance of 2.5 meters, and arrows and safety warnings are imaged at 7 meters. However, as the core of this research focuses on the elemental image (EI) generation technology for the light-field display itself, the image generation depths discussed subsequently refer to millimeter-scale distances within the microlens array (MLA) optical path, rather than the final imaging distances after projection by the mirror.

2. Dynamic Path Analysis:

The initial stage of the proposed pipeline involves automated trajectory extraction and multi-depth layer assignment. A skeletonization-based algorithm is utilized to parse navigation paths from input imagery, enabling the dynamic determination of directional arrow positions. By integrating these navigation cues with safety warning frames at predefined imaging distances, the system generates a set of structured input layers. Each layer is characterized by specific spatial coordinates and longitudinal depth parameters, serving as the foundational input for the subsequent lens-based CGEI synthesis engine.

3. Lens-Based EI Pickup Model

3.1 Geometric Model

The proposed system adopts a standard integral imaging configuration comprising a high-resolution display panel and a microlens array (MLA). The fundamental geometric relationship is governed by the lens pitch P_{EL} defined as the center-to-center distance between adjacent microlenses, and the pixel pitch P_s representing the physical spacing of neighboring display pixels.

The MLA is positioned at a distance g from the display panel, with each microlens possessing a focal length f . The number of pixels covered by a single microlens along one axis is defined in Eq. (1):

$$P = \frac{P_{EL}}{P_s} \quad (1)$$

Where P defines the tile size utilized in the subsequent patch mapping process. This parameter, along with the projection to the target depth z , constitutes the physical basis for the proposed lens-based pickup model. A schematic illustration of the optical geometry, adapted from Kwon et al. [2], is shown in Fig. 1.

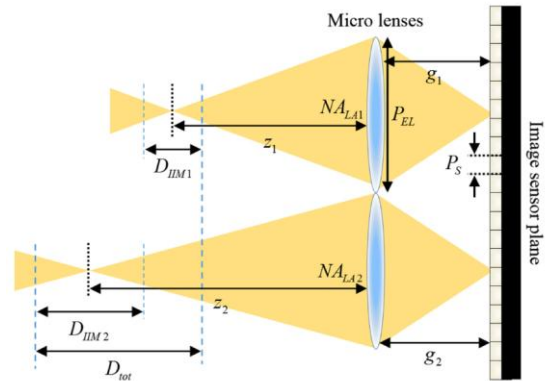


Figure 1. Schematic illustration of the optical geometry. [2]

3.2 Geometric Parameter Derivation

For each scene layer located at depth z , the geometric expansion of its projection through the microlens array is modeled by an extension parameter, denoted as $E(z)$. Based on the MLA pitch and the thin-lens approximation, $E(z)$ expressed in pixel units is defined in Eq. (2):

$$E(z) = \frac{P_{EL} \cdot (z/f)}{P_s} \quad (2)$$

Where P_{EL} is the microlens pitch, f is the microlens focal length, and P_s is the display pixel pitch. This quantity represents the depth-dependent lateral expansion of the elemental image (EI) footprint on the display plane. Utilizing this extension parameter, a depth-dependent scaling factor, $S(z)$, is derived to govern the template resizing process, $S(z)$ is derived in Eq. (3):

$$S(z) = \frac{P}{E(z) + \frac{P_{EL}}{2 \cdot P_S}} \quad (3)$$

Where P denotes the tile size (number of pixels covered by each microlens). Note that the term $P_{EL}/2$ is normalized by P_S to maintain unit consistency in the pixel domain. The corresponding resolution of the processed template for the layer at depth z is then given by $N'_x(z)$ and $N'_y(z)$ defined in Eq. (4):

$$N'_x(z) = N_x \cdot S(z), N'_y(z) = N_y \cdot S(z) \quad (4)$$

Where N_x and N_y represent the native horizontal and vertical resolutions of the display, respectively.

3.3 Template Construction

For each depth layer z a depth-aware template image denoted as $T(z)$, is generated and subsequently utilized for mapping across all microlens tiles. The construction process comprises three sequential steps: depth-dependent resizing, optical inversion, and boundary padding.

3.3.1 Depth-dependent resizing

The template $T(z)$ is first initialized by resizing the raw input image to the target resolution $(N'_x(z), N'_y(z))$ derived in Eq. (4)

This resizing operation ensures the correct magnification of the scene layer, matching the effective acquisition range focused at depth z .

3.3.2 Optical inversion

Following resizing, the template undergoes a centrosymmetric inversion (horizontal and vertical flip), effectively producing a 180° rotation. This step compensates for the inherent image reversal introduced by the real image formation of the microlens array, ensuring that the spatial orientation of each extracted patch matches the physical optical acquisition model.

3.3.3 Boundary padding

During the subsequent EI synthesis, each microlens extracts a patch of size $P \times P$ from $T(z)$. As the viewing angle shifts across the array, the sampling window moves laterally. To guarantee valid sampling for all microlenses—particularly those near the display boundaries—the template is symmetrically zero-padded. The required padding width $L_{pad}(z)$, is computed based on the extension parameter and the scaling factor defined in Eq. (5):

$$L_{pad}(z) = E(z) \cdot S(z) \quad (5)$$

This padding ensures that every microlens can access a complete $P \times P$ region from the template without boundary artifacts, providing the necessary spatial buffer for the tile-based synthesis described in Section 3.4.

3.4 Tile-Based EI Synthesis

After generating the depth-aware templates $\{T(z)\}$, the final elemental image, denoted as EI , is synthesized on the display plane using a tile-based assembly strategy. Each tile corresponds to a square region of $P \times P$ pixels covered by a single microlens. Given the microlens pitch corresponds to P display pixels, the EI is spatially partitioned at discrete intervals defined in Eq. (6):

$$i = 0, P, 2P, \dots, N_x - P; \quad j = 0, P, 2P, \dots, N_y - P \quad (6)$$

For each depth layer z , the sampling origin (x_s, y_s) on the template (T_z) for a specific tile (i, j) is determined by a projection-based geometric shift. Substituting the extension parameter $E(z)$ and scaling factor $S(z)$, the starting coordinates

for patch cropping are computed defined in Eq. (7) and Eq. (8):

$$x_s = \left(\frac{N_x - i}{N_x} \right) \left(N_x + \frac{E(z)}{2} \right) S(z) \quad (7)$$

$$y_s = \left(\frac{N_y - j}{N_y} \right) \left(N_y + \frac{E(z)}{2} \right) S(z) \quad (8)$$

These expressions mathematically model the geometric parallax shift of the scene layer as projected through the microlens array onto the display plane. For each tile iteration, a patch $P_{i,j}(z)$ of size $P \times P$ is extracted from the template defined in Eq. (9):

$$P_{i,j}(z) = T(z)[x_s : x_s + P, y_s : y_s + P] \quad (9)$$

The extracted patch is then merged into the accumulation buffer EI . To preserve the visual information from all depth layers, we employ a logical union operation, ensuring that non-zero pixels from the current patch are retained in the final image defined in Eq. (10):

$$EI[i : i + P, j : j + P] \leftarrow EI[i : i + P, j : j + P] \cup P_{i,j}(z) \quad (10)$$

This accumulation process is iterated over all tile coordinates (i, j) and all depth layers z , yielding the final full-resolution EI (e.g., 2160×2160 pixels).

4. GPU-Parallelized EI Synthesis Pipeline

4.1 Pixel-Parallel Architecture

Unlike the CPU implementation which iterates through microlens tiles sequentially, our GPU framework adopts a fine-grained parallel strategy. We map the compute grid directly to the display resolution $(N_x \times N_y)$, assigning a single CUDA thread to each physical pixel of the elemental image (e.g., 2160×2160 threads). Each thread independently identifies its corresponding microlens tile index (i, j) and executes the depth-merging logic. This design completely eliminates the overhead of nested loops on the host side and ensures that the GPU compute cores are fully saturated.

4.2 LUT-Driven Geometric Access

To eliminate runtime arithmetic overhead, we pre-calculate all depth-dependent spatial parameters—including extension factors $E(z)$, and tile sampling offsets—on the CPU and upload them to the GPU as Look-Up Tables (LUTs). This approach replaces costly floating-point operations (multiplication, division, and rounding) within the kernel with efficient global memory fetches. By decoupling geometric derivation from pixel rendering, the kernel workload is streamlined into a branchless lookup process, maximizing instruction throughput.

4.3 Unified Mask Stack Strategy

To mitigate Host-to-Device transfer bottlenecks, we implement a Unified Mask Stack architecture. Multi-depth RGB templates are first binarized and packed into a contiguous 3D array $(N_{layer} \times H \times W)$ on the host, effectively reducing memory bandwidth requirements by threefold. The entire stack is transferred to the GPU in a single transaction prior to execution. The kernel then performs a logical union across depth layers using this pre-loaded data, enabling the single-pass synthesis of high-resolution, multi-depth EIs without iterative memory copying.

5. Result & Discussion

5.1 Experimental Result

As shown in Fig. 2, the reference driving environment depicts a curved road scenario for operational evaluation. In addition to dashboard telemetry, the pipeline dynamically extracts navigation trajectories for arrow placement and identifies the oncoming vehicle as a safety-critical target. It provides the necessary context to verify the system's ability to synthesize cues at multiple imaging distances, ensuring alignment with the requirements of real-world automotive scenarios.



Figure 2. Simulated operational scenario for automotive AR HUD systems.

In Fig. 3, we present the optical simulation results generated using LightTools software, corroborating the system's capability to fulfill the operational scenario requirements described above. By numerically focusing on objects at varying depths—specifically the dashboard, navigation arrows, and the safety warning frame—the results demonstrate distinct spatial separation and clear image reconstruction at the target imaging distances. The corresponding experimental setup and detailed hardware specifications utilized for this simulation are listed in Table 1.

Table 1. Detailed experimental setup and hardware specifications

Image parameters	
Resolution (#)	2000×2000
Pixel size (mm)	0.024
Lens array parameters	
Focal length (mm)	1
Pitch (mm)	0.36
Lens Number (#)	144×144
Panel parameters	
Resolution / lens	15×15
Resolution (#)	2160×2160
Pixel size (mm)	0.024
Screen size (mm)	51.84
Hardware specification	
CPU	AMD Ryzen 9 9950X 16-core
GPU	NVIDIA GeForce RTX 5090

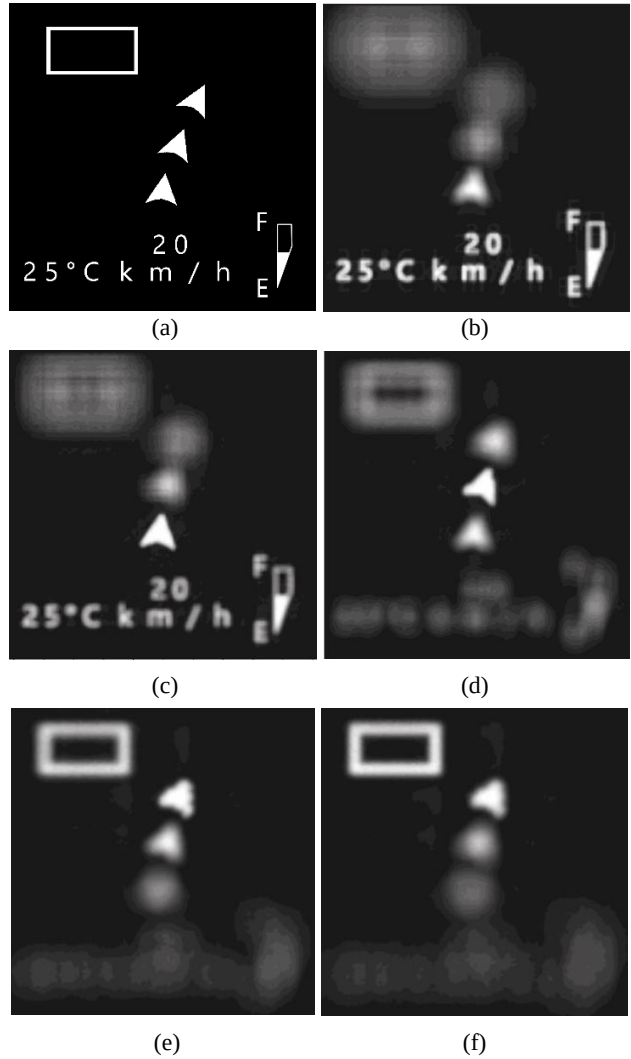


Figure 3. (a) The original 2D composite image used as the system input. (b) Focused on the dashboard interface ($z = 5$ mm). (c) Focused on the bottom arrow ($z = 10$ mm). (d) Focused on the middle arrow ($z = 20$ mm). (e) Focused on the top arrow ($z = 30$ mm). (f) Focused on the warning frame ($z = 35$ mm)

5.2 Computational Performance Analysis

To evaluate operational efficiency, we benchmarked the proposed lens-based framework against a conventional GPU-accelerated pixel-wise ray tracing algorithm [3] across single- and multi-depth scenarios. We adopted End-to-End Latency as the primary metric, which comprehensively captures the entire execution pipeline: CPU-based path analysis, Host-to-Device (H2D) memory transfer, GPU kernel execution, and Device-to-Host (D2H) output synchronization. Unlike prior evaluations that often exclude system overhead, this approach reflects realistic automotive computing constraints.

In Fig. 4, we utilized a standardized benchmarking suite to evaluate the system's sensitivity to depth variations. For single-depth scenarios, source imagery was positioned at discrete planes ($z=10, 20, 30$, and 35 mm) to isolate the computational impact of depth-dependent scaling and sampling shifts within the LUT-driven architecture. By explicitly measuring the temporal contribution of

each pipeline stage—including trajectory extraction and multi-depth structure encapsulation—we ensure that performance gains are attributed to the systemic optimization of the Unified Mask Stack rather than reduced content complexity. The comprehensive computational results are summarized in Table 2.

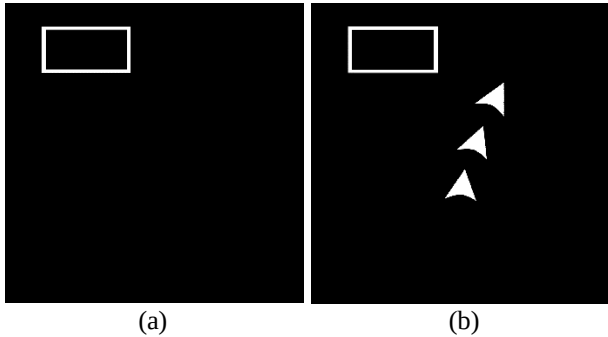


Figure 4. (a) Single-depth scenarios evaluated at discrete depths of $z = 10, 20, 30$, or 35 mm. (b) Multi-depth scenario comprising spatial layers, arrows at $z = 10, 20, 30$ mm and a warning frame at $z = 35$ mm.

Table 2. Computation time comparison for single- and multi-depth scenarios

Single-Depth Generation (second)			
Depth	Pixel-Wise Ray Tracing [3] (GPU)	Proposed Lens-Based (CPU)	Proposed Lens-Based (GPU)
$Z = 10$ mm	0.5293	0.1865	0.0303
$Z = 20$ mm	0.5559	0.1863	0.0309
$Z = 30$ mm	0.5612	0.1875	0.0320
$Z = 35$ mm	0.5914	0.1916	0.0322
Multi-Depth Generation (second)			
Pixel-Wise Ray Tracing [3] (GPU)		Proposed Lens-Based (CPU)	Proposed Lens-Based (GPU)
0.6378		0.6509	0.0320

5.3 Image Quality Assessment

To quantify the fidelity of the light-field reproduction, we performed computational reconstruction from the GPU-generated elemental images at three distinct depths ($z = 10, 20$ and 30 mm). These reconstructed views were compared against the original source images to evaluate how effectively the proposed lens-based algorithm preserves structural information during encoding. The resulting reconstructed images are illustrated in Fig. 5. Correspondingly, the quantitative performance metrics, including the specific PSNR and SSIM values for each depth layer, are summarized in Table 3.

Table 3. Quantitative assessment of reconstruction quality (PSNR and SSIM) across varying depths.

Depth	SSIM	PSNR
$Z = 10$ mm	0.9635	19.0761
$Z = 20$ mm	0.9378	19.0817
$Z = 30$ mm	0.9179	19.0819

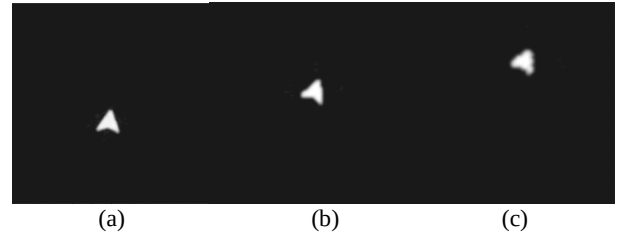


Figure 5. Reconstructed results of the arrow test pattern corresponding to varying depths used for quality assessment. (a) $z = 10$ mm (b) $z = 20$ mm (c) $z = 30$ mm.

6. Discussion

Table 2 demonstrates substantial performance gains within the proposed End-to-end pipeline. The framework achieves an average latency (calculated over 10 consecutive frames) of <0.033 s for single-depth and 0.032 s for multi-depth scenarios, achieving a $>19\times$ speedup over the conventional GPU method (0.6378 s). The Unified Mask Stack ensures stable millisecond-level performance independent of depth complexity. Regarding quality (Table 3), SSIM consistently exceeds 0.91 . The relatively lower PSNR is attributed to aliasing artifacts inherent in the discrete template resizing process; specifically, the interpolation used during scaling amplifies edge jaggedness, resulting in pixel-level deviations at high-frequency boundaries. However, high SSIM scores confirm that global structural fidelity and essential depth cues are accurately preserved.

7. Conclusion

We proposed a dynamic, lens-based CGEI synthesis pipeline leveraging skeletonization-based trajectory extraction and a Unified Mask Stack to eliminate pixel-wise ray tracing overhead. The implemented GPU-parallelized system generates high-resolution (2160×2160) multi-depth EIs with an average latency of 0.032 s, achieving a $>19\times$ speedup over conventional methods while maintaining robust structural fidelity ($SSIM > 0.91$). These results validate the framework's potential as a viable rendering engine for achieving approaching real-time performance in light-field automotive HUDs.

8. References

1. Zhu Y, Sang Xinzhong, Yu Xunbo, Wang Peng, Xing Shujun, Chen Duo, et al. Wide field of view tabletop light field display based on piece-wise tracking and off-axis pickup. *Optics Communications*. 2017 Nov;402:41–46.
2. Kwon K-C, Erdenebat M-U, Alam MdA, Lim Y-T, Kim KG, Kim N. Integral imaging microscopy with enhanced depth-of-field using a spatial multiplexing. *Optics Express*. 2016;24(3):2072–2083.
3. Chen EH. Accelerated algorithm for computer generated light field displays based on integral imaging. Master's thesis. Taipei, Taiwan: National Taiwan University. 2025.